PKG 15: ENUM, DICTIONARY, GOOGLE GUAVA, and HASH CODE

```
/**
 * @author Oracle
 */
public class EnumDay {

    private enum EnumDaySet {
        SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY
    }

    EnumDaySet day;

    public EnumDay(EnumDaySet day) {
        this.day = day;
    }

    public void tellItLikeItIs() {
        switch (day) {
            case MONDAY:
                System.out.println("Mondays are bad.");
                break;

            case FRIDAY:
                System.out.println("Fridays are better.");
                break;

            case SATURDAY:
            case SUNDAY:
                System.out.println("Weekends are best.");
                break;

            default:
                System.out.println("Midweek days are so-so.");
                break;
        }
    }

    public static void main(String[] args) {
        EnumDay firstDay = new EnumDay(EnumDaySet.MONDAY);
        firstDay.tellItLikeItIs();

        EnumDay thirdDay = new EnumDay(EnumDaySet.WEDNESDAY);
        thirdDay.tellItLikeItIs();

        EnumDay fifthDay = new EnumDay(EnumDaySet.FRIDAY);
        fifthDay.tellItLikeItIs();

        EnumDay sixthDay = new EnumDay(EnumDaySet.SATURDAY);
        sixthDay.tellItLikeItIs();

        EnumDay seventhDay = new EnumDay(EnumDaySet.SUNDAY);
        seventhDay.tellItLikeItIs();
    }
}

/* OUTPUT
Mondays are bad.
Midweek days are so-so.
Fridays are better.
Weekends are best.
Weekends are best.
*/
```

ENUM

All the objects are predefined and static
The words are uppercase because the references are final

```
64  public enum CSCoreClasses {
65      CSC210(1, "Introduction to Computer Programming", "CSC 210", 3),
66      CSC220(2, "Data Structures", "CSC 220", 3),
67      CSC340(3, "Programming Methodology", "CSC 340", 3),
68      CSC413(4, "Software Development", "CSC 413", 3),
69      CSC600(5, "Programming Language Design", "CSC 600", 3),
70      CSC000();
71      private final String GENERAL_NOTE = "SFSU 2018 Catalog";
72      private int cNum;
73      private String cDesc;
74      private String cTittle;
75      private int cNumUnits;
76
77      private CSCoreClasses() {
78          this.cTittle = "CSC000";
79      }
80
81      private CSCoreClasses(int cNum, String cDesc, String cTittle, int cNumUnits) {
82          this.cNum = cNum;
83          this.cDesc = cDesc;
84          this.cTittle = cTittle;
85          this.cNumUnits = cNumUnits;
86      }
87
88      public int getcNum() {
89          return cNum;
90      }
91      public void setcNum(int cNum) {
92          this.cNum = cNum;
93      }
94
95      public String getcDesc() {
96          return cDesc;
97      }
98      public void setcDesc(String cDesc) {
99          this.cDesc = cDesc;
100     }
101
102     public String getcTittle() {
103         return cTittle;
104     }
105     public void setcTittle(String cTittle) {
106         this.cTittle = cTittle;
107     }
108
109     public int getcNumUnits() {
110         return cNumUnits;
111     }
112     public void setcNumUnits(int cNumUnits) {
113         this.cNumUnits = cNumUnits;
114     }
115
116     @Override
117     public String toString() {
118         return this.cNum + ", " + this.cDesc + ", "
119                 + this.cTittle + ", " + this.cNumUnits;
120     }
121
122
```

List of objects is a final list, we cannot add more once we compile it

- enum
- Data(capital stuff)
- what to do with the data is declared later

Names are unique, they cannot be duplicates

Start from line #75 for asmt 5

Text

ENUM

```
public static void main(String[] args) {

        System.out.println("valueOf()----------------------------------------");
        CSCoreClasses CSC999 = CSCoreClasses.valueOf("CSC220");
        CSC220.setcNumUnits(4);
        CSC999.setcTittle(CSC220.cTittle + "-999");
        System.out.println(CSC999);
        System.out.println(CSC220);

        System.out.println("values()----------------------------------------");
        System.out.println(CSCoreClasses.values().length); // 6
        for (CSCoreClasses entry : CSCoreClasses.values()) {
            System.out.println(entry);
        }

        System.out.println("static----------------------------------------");
        System.out.println(CSCoreClasses.CSC210.GENERAL_NOTE);
        System.out.println(CSCoreClasses.CSC210.cDesc);
        System.out.println(CSCoreClasses.CSC210);

        System.out.println("Constructor-------------------------------------");
        System.out.println(CSCoreClasses.CSC000.GENERAL_NOTE);
        System.out.println(CSCoreClasses.CSC000.cDesc);
        System.out.println(CSCoreClasses.CSC000);

        System.out.println("name()----------------------------------------");
        System.out.println(CSC999.name());
        System.out.println(CSC210.name().length());

        System.out.println("equals() and compareTo()------------------------");
        System.out.println(CSC210.equals(CSC210));      // true
        System.out.println(CSC210.equals(CSC220));      // false
        System.out.println(CSC210.compareTo(CSC220));   // -1
        System.out.println(CSC210.compareTo(CSC340));   // -2
        System.out.println(CSC210.compareTo(CSC413));   // -3
        System.out.println(CSC210.compareTo(CSC600));   // -4

        System.out.println("equals() and hashCode()-------------------------");
        String a = "Mickey";
        String b = a;
        System.out.println(b.hashCode() + " " + a.hashCode()); // Same integer

        System.out.println("ordinal() and hashCode()-----------------------");
        System.out.println(CSC210.ordinal() + " " + CSC210.hashCode()); // 0 ...
        System.out.println("  " + System.identityHashCode(CSC210));// 865113938
        System.out.println(CSC220.ordinal() + " " + CSC220.hashCode()); // 1 ...
        System.out.println("  " + System.identityHashCode(CSC220));// 1442407170
        System.out.println(CSC340.ordinal() + " " + CSC340.hashCode()); // 2 ...
        System.out.println("  " + System.identityHashCode(CSC340));// 1028566121
        System.out.println(CSC413.ordinal() + " " + CSC413.hashCode()); // 3 ...
        System.out.println("  " + System.identityHashCode(CSC413));// 1118140819
        System.out.println(CSC600.ordinal() + " " + CSC600.hashCode()); // 4 ...
        System.out.println("  " + System.identityHashCode(CSC600));// 1975012498
    }
}
```

ENUM

```
/* OUTPUT

valueOf()-------------------------------------
2, Data Structures, CSC 220-999, 4
2, Data Structures, CSC 220-999, 4

values()--------------------------------------
6
1, Introduction to Computer Programming, CSC 210, 3
2, Data Structures, CSC 220-999, 4
3, Programming Methodology, CSC 340, 3
4, Software Development, CSC 413, 3
5, Programming Language Design, CSC 600, 3
0, null, CSC000, 0

static----------------------------------------
SFSU 2018 Catalog
Introduction to Computer Programming
1, Introduction to Computer Programming, CSC 210, 3

Constructor-----------------------------------
SFSU 2018 Catalog
null
0, null, CSC000, 0

name()----------------------------------------
CSC220
6

equals() and compareTo()----------------------
true
false
-1
-2
-3
-4

equals() and hashCode()-----------------------
-1990497576 -1990497576

ordinal() and hashCode()----------------------
0 865113938
  865113938
1 1442407170
  1442407170
2 1028566121
  1028566121
3 1118140819
  1118140819
4 1975012498
  1975012498

*/
```

DICTIONARY and HASH CODE

```java
import java.util.Dictionary;
import java.util.Enumeration;
import java.util.Hashtable;

public class TinyBook {
    public static void main(String[] args) {
        Dictionary tinyBook = new Hashtable();

        // Adding some key/value
        System.out.println("Adding----------------------------------------");
        tinyBook.put("book", "A set of pages.");
        tinyBook.put("book", "To arrange something on a particular date.");
        tinyBook.put("bookable", "Can be ordered.");
        tinyBook.put("bookbinder", "A person who fastens the pages of books.");
        tinyBook.put("bookcase", "A piece of furniture with shelves.");

        // Getting the size
        System.out.println("Size-----------------------------------------");
        System.out.println(tinyBook.size()); // OUTPUT: 4

        // Getting values
        System.out.println("Values---------------------------------------");
        String book = (String) tinyBook.get("book");
        System.out.println(book);
        // OUTPUT: To arrange something on a particular date.

        // Iterating over entire Mutlimap
        System.out.println("Iterate--------------------------------------");
        for (Enumeration entry = tinyBook.elements(); entry.hasMoreElements();) {
            System.out.println(entry.nextElement());
        }
        /* OUTPUT:      A person who fastens the pages of books.
                        To arrange something on a particular date.
                        Can be ordered.
                        A piece of furniture with shelves.                    */

        // Getting hash codes
        System.out.println("Hash Code------------------------------------");
        for (Enumeration entry = tinyBook.elements(); entry.hasMoreElements();) {
            System.out.println(System.identityHashCode(entry.nextElement()));
        }/* OUTPUT:     865113938
                        1442407170
                        1028566121
                        1118140819                                            */

        System.out.println("Hash Code------------------------------------");
        System.out.print(tinyBook.hashCode() + "\t");
        System.out.println(System.identityHashCode(tinyBook));
        // OUTPUT: -1631487136      1975012498

        // Removing a single value
        System.out.println("Remove---------------------------------------");
        tinyBook.remove("bookcase");
        System.out.println(tinyBook.get("bookcase")); // null

        // Remove all values for a key
        System.out.println("Remove All-----------------------------------");
        tinyBook.remove("book");
        System.out.println(tinyBook.get("book")); // null
    }
}
```

GOOGLE GUAVA LIBRARY and HASH CODE

```
import java.util.Collection;
import com.google.common.collect.ArrayListMultimap;
import com.google.common.collect.Multimap;

public class TinyBookGoogleGuava {

    public static void main(String[] args) {

        Multimap<String, String> tinyBook = ArrayListMultimap.create();

        // Adding some key/value
        System.out.println("Adding----------------------------------------");
        tinyBook.put("book", "A set of pages.");
        tinyBook.put("book", "To arrange something on a particular date.");
        tinyBook.put("bookable", "Can be ordered.");
        tinyBook.put("bookbinder", "A person who fastens the pages of books.");
        tinyBook.put("bookcase", "A piece of furniture with shelves.");

        // Getting the size
        System.out.println("Size-----------------------------------------");
        System.out.println(tinyBook.size());  // 5

        // Getting values
        System.out.println("Values---------------------------------------");
        Collection<String> book = tinyBook.get("book");
        System.out.println(book);
        // [A set of pages., To arrange something on a particular date.]

        Collection<String> bookcase = tinyBook.get("bookcase");
        System.out.println(bookcase);
        // [A piece of furniture with shelves.]

        // Iterating over entire Mutlimap
        System.out.println("Iterate--------------------------------------");
        tinyBook.values().forEach((value) -> {
            System.out.println(value);
        });

        // Getting hash codes
        System.out.println("Hash Code------------------------------------");
        tinyBook.entries().forEach((entry) -> {
            System.out.println(entry.hashCode() + "\t"
                    + System.identityHashCode(entry));
        });

        System.out.println("Hash Code------------------------------------");
        System.out.print(tinyBook.hashCode() + "\t");
        System.out.println(System.identityHashCode(tinyBook));

        // Removing a single value
        System.out.println("Remove---------------------------------------");
        tinyBook.remove("bookcase", "A piece of furniture with shelves.");
        System.out.println(tinyBook.get("bookcase")); // []

        // Remove all values for a key
        System.out.println("Remove All-----------------------------------");
        tinyBook.removeAll("book");
        System.out.println(tinyBook.get("book")); // []
    }
}
```

```
GOOGLE GUAVA LIBRARY and HASH CODE

Adding----------------------------------------

Size------------------------------------------
5

Values----------------------------------------
[A set of pages., To arrange something on a particular date.]
[A piece of furniture with shelves.]

Iterate---------------------------------------
Can be ordered.
A set of pages.
To arrange something on a particular date.
A person who fastens the pages of books.
A piece of furniture with shelves.

Hash Code-------------------------------------
-920681491  625576447
1244648863  1560911714
-148441025  939047783
-323823247  1237514926
-238541373  548246552

Hash Code-------------------------------------
-1794801272 401625763

Remove----------------------------------------
[]

Remove All------------------------------------
[]
```

Google Guava

From Wikipedia, the free encyclopedia

Google Guava is an open-source set of common libraries for Java, mainly developed by Google engineers.

Google Guava	
Original author(s)	Kevin Bourrillion and Jared Levy (Google Collections Library) [1]
Developer(s)	Google
Stable release	26.0 / August 2, 2018[2]
Repository	github.com/google/guava
Written in	Java
Operating system	Cross-platform
Type	Utility and Collection Libraries
License	Apache License 2.0
Website	github.com/google/guava

Contents [hide]

1 Overview
2 Controversy
3 See also
4 References
5 External links

Overview [edit]

Google Guava can be roughly divided into three components: basic utilities to reduce menial labors to implement common methods and behaviors; an extension to the Java collections framework (JCF) formerly called the **Google Collections Library**; and other utilities which provide convenient and productive features such as functional programming, graphs, caching, range objects, and hashing.[3]

The creation and architecture of the collection component were partly motivated by generics introduced in JDK 1.5.[1] Although generics improve the productivity of programmers, the standard JCF does not provide sufficient functionality, and its complement Apache Commons Collections had not adopted generics in order to maintain backward compatibility.[1] This fact led two engineers Kevin Bourrillion and Jared Levy to develop an extension to JCF, which provides additional generic classes such as multisets, multimaps, bimaps, and immutable collections.[1]

The library's design and code were advised and reviewed by Joshua Bloch, the original lead designer of the Java Collections framework, and Doug Lea, one of the lead designers of concurrency utilities in JDK.[1]

As of April 2012, Guava ranked the 12th most popular Java library, next to the Apache Commons projects and a few others.[4] Research performed in 2013 on 10,000 GitHub projects found that Google-made libraries, such as Google Web Toolkit and Guava, constituted 7 of the top 100 most popular libraries in Java, and that Guava was the 8th most popular Java library.[5] As of March 2018, Guava was the 6th most starred Java project on Github.[6]